



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验三：内核线程与缺页异常

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 了解进程与线程的概念、相关结构和实现。
2. 实现内核线程的创建、调度、切换。(栈分配、上下文切换)
3. 了解缺页异常的处理过程，实现进程的栈增长。

二. 实验内容

1. 进程模型设计

1.1. 进程控制块

在实验中使用 **Process** 结构体表示一个进程，它含有 **pid** 和 **inner** 两个字段，分别表示进程的 ID 和内部数据。**inner** 字段是一个 **RwLock** 类型的字段，它包含了一个 **ProcessInner** 结构体，这个结构体包含了进程的其他信息，包括进程的状态、调度计次、退出返回值、内存空间、父子关系、中断上下文、文件描述符表等等

1.2. 进程上下文

在抢占式操作系统中，进程的调度是通过中断来实现的。当一个进程的时间片用完后，操作系统会触发一个时钟中断，进程调度器会被唤醒，它会根据进程的状态和调度策略来决定下一个要运行的进程。在调度器决定好下一个要运行的进程后，它会将当前进程的上下文保存起来，然后将下一个进程的上下文恢复，从而使得下一个进程得以运行。

在之前的实验中，已经描述过在 **x86_64** 架构下的中断发生时，CPU 会将当前的一部分上下文保存到内核栈中，然后跳转到中断处理函数。这些上下文包括：

instruction_pointer: 指令指针，保存了中断发生时 CPU 正在执行的指令的地址。
code_segment: 代码段寄存器，保存了当前正在执行的代码段的选择子。
cpu_flags: CPU 标志寄存器，保存了中断前的 CPU 标志状态。
stack_pointer: 栈指针，保存了中断前的栈指针。
stack_segment: 栈段寄存器，保存了中断前的栈段选择子，在 **x86_64** 下总是为 0。而在进行进程切换时，通常还需要保存和恢复更多的上下文，这些内容主要包括通用寄存器和浮点寄存器。为了简化实现，实验在编译架构选项中禁用了浮点寄存器，因此只需要保存和恢复通用寄存器即可。

1.3. 进程页表

进程的页表是通过向 Cr3 寄存器写入页表的物理地址来实现控制的,因此在进程切换时,需要将进程的页表物理地址写入 Cr3 寄存器。

除了内核进程的页表在启动时被初始化外,其他进程的页表都是在进程创建时被初始化的。它们通常通过克隆内核进程来实现,这样做的目的是当程序陷入中断时,CPU 能够正常访问到内核的代码和数据,从而能够正常的进行系统调用。

1.4. 进程调度

进程调度的工作目的简单来说就是从就绪队列中选取一个进程,并将其分配给选定的 CPU 核心运行。在实验中,为了实现简单,YSOS 自始至终只会使用一个 CPU 核心。

在实验中,将实现一个简单的 FIFO 调度器,它会将就绪队列中的第一个进程选出来,然后将其分配给 CPU 核心运行,当进程的时间片用完后,调度器会将其重新放回就绪队列的末尾。

1.4.1. 内核进程

在初始化进程管理器时,为了让内核始终和其他进程有一样的获取时间片的机会,需要将内核进程作为第一个进程加入进程列表。

之后的每次切换时,内核进程也会被视作一个普通的进程,从而被调度器选中、保存、执行。

1.4.2. 时钟中断的处理

时钟中断发生时,进程调度器被运行,CPU 通过 IDT 调用中断处理程序,完成进程的切换。

在一次进程切换的过程中,需要关闭中断,之后完成以下几个步骤:

- 保存当前进程的上下文

经过上述进程上下文的描述可知,作为可变引用参数被传入的 `ProcessContext` 中存储了进程切换时需要保存的所有寄存器的值,并且其中的内容将会在进程切换完成后被恢复,进而真正实现进程的切换。

因此,进程切换的第一步就是将 `context` 保存至当前进程的 `ProcessInner` 中,以便下次恢复运行状态。

- 更新当前进程的状态

进程切换时,若它当前的状态并非 `Dead`,则当前进程的状态会被更新为 `Ready`。

同时,为了记录进程的执行时间,一般也会记录进程的调度次数,这里使用 `usize` 类型的 `ticks_passed` 字段来记录进程的调度次数。

- 将当前进程放入就绪队列

当前进程被切换出去后,它会被放入就绪队列的末尾,等待下一次调度。

- 从就绪队列中选取下一个进程

进程调度器会从就绪队列中选取第一个进程,检查进程的状态,如果进程处于可调度状态,就将其状态更新为 `Running`,并将其 `PID` 写入 `Processor` 中。

- 切换进程上下文和页表

进程调度器会将选中的进程的上下文重新加载,并将新进程的页表物理地址写入 Cr3 寄存器,从而完成进程的切换。

1.5. 进程的内存布局

在开启了虚拟内存的情况下，操作系统拥有巨大的地址空间，可以赋予它们不同的功能，管理这些内存如何使用。

在已经进行过的实验中，笔者为大家预设了一些内存布局，你应该或多或少接触过这些地址：

- 物理内存偏移：0xFFFF800000000000

通过定义物理内存偏移，借助 2MB 的页映射，将物理内存线性映射到了这一偏移量所对应的地址空间中。在内核中，当需要访问一个物理内存地址，如 0x1000 时，就可以通过 0xFFFF800000001000 来访问。

- 内核空间起始地址：0xFFFFFFFF0000000000

在操作系统中，一般会将内核地址映射到高偏移，而将用户地址映射到低偏移。在实验中，内核空间的起始地址被定义在了 0xFFFFFFFF0000000000，这相当于为内核预留了 1TiB 的地址空间。

通过 kernel.ld 链接器脚本，将内核的起始地址设置为了这一地址，实验中编译的操作系统内核会被链接到从这一地址开始的地址空间中。同时，通过 bootloader 中的内核加载函数，读取 ELF 文件的描述，并将这些内容加载到了对应的地址空间中。

- 内核栈地址：0xFFFFFFFF0100000000

内核栈的起始地址通过配置文件被定义在了 0xFFFFFFFF0100000000，距离内核起始地址 4GiB。默认大小为 512 个 4KiB 的页面，即 2MiB。

在虚拟内存的规划中，任意进程的栈地址空间大小为 4GiB。以内核为例，内核栈所对应的内存区域的起始地址为 0xFFFFFFFF0100000000，结束地址为 0xFFFFFFFF0200000000。

2. 合并实验代码

3. 进程管理器的初始化

4. 进程调度的实现

5. 进程信息的获取

5.1. 环境变量

5.2. 进程返回值

6. 内核线程的创建

7. 缺页异常的处理

8. 进程的退出

三. 实验过程

1. 合并实验代码

我在 lab2 的基础上新建了 lab3 分支，然后将官方文档下载到根目录下的 `src/0x03/crates/kernel` 文件夹中，再在 YatSenOS 根目录下运行以下命令：

```
cp -rf src/0x03/crates/kernel/* yatsenos/
```

这样就完成了实验代码的合并。

2. 进程管理器的初始化

在 `src/proc/mod.rs` 中，补全 `init` 函数，这个函数将内核包装成进程，并将其传递给 `ProcessManager`，使其成为第一个进程。

```

/// init process manager
pub fn init(boot_info: &'static boot::BootInfo) {
    let proc_vm = ProcessVm::new(PageTableContext::new()).init_kernel_vm();

    trace!("Init kernel vm: {:#?}", proc_vm);

    let mut proc_data = ProcessData::new();

    // 从 boot.conf 配置中获取的信息（硬编码或从配置解析）
    proc_data.set_env("KERNEL_STACK_ADDR", "0xFFFFFFFF0100000000");
    proc_data.set_env("KERNEL_STACK_SIZE", "512");
    proc_data.set_env("KERNEL_PATH", "\\KERNEL.ELF");

    // 从 BootInfo 获取的信息
    proc_data.set_env("PHYSICAL_MEM_OFFSET", format!("{:#x}",
boot_info.physical_memory_offset).as_str());
    proc_data.set_env("SYSTEM_TABLE", format!("{:#x}",
boot_info.system_table.as_ptr() as u64).as_str());

    // 从运行时获取的信息
    proc_data.set_env("KERNEL_HEAP_SIZE", format!("{}", HEAP_SIZE).as_str());

    // kernel process
    let kproc = {
        /* FIXME: create kernel process */
        Process::new(
            String::from("kernel"),
            None,
            Some(proc_vm),
            Some(proc_data),
        )
    };
    manager::init(kproc);

    info!("Process Manager Initialized.");
}

```

3. 进程调度的实现

3.1. 修改时间中断的内容

```

use super::consts::*;
// 引入原子类型和内存排序规则
use core::sync::atomic::{AtomicU64, Ordering};
use x86_64::structures::idt::{InterruptDescriptorTable, InterruptStackFrame};
use crate::as_handler;
use crate::proc::{switch, ProcessContext};
use crate::memory::gdt::CLOCK_IST_INDEX;

as_handler!(clock);
pub unsafe fn register_idt(idt: &mut InterruptDescriptorTable) {
    idt[Interrupts::IrqBase as u8 + Irq::Timer as u8]
        .set_handler_fn(clock_handler)
        .set_stack_index(CLOCK_IST_INDEX);
}

static COUNTER: AtomicU64 = AtomicU64::new(0);

#[inline]
pub fn read_counter() -> u64 {
    // 使用 Relaxed 内存序读取当前值
    COUNTER.load(Ordering::Relaxed)
}

#[inline]
pub fn inc_counter() -> u64 {
    // fetch_add 会将变量加 1, 并返回相加【之前】的旧值。
    // 为了让 inc_counter 返回最新的值, 我们在后面 + 1
    COUNTER.fetch_add(1, Ordering::Relaxed) + 1
}

pub fn clock(context: &mut ProcessContext) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        /*if inc_counter() % 0x100 == 0 {
            info!("Tick! @{}", read_counter());
        }*/
        switch(context);
        super::ack();
    });
}

```

修改时间中断函数, 在 TSS 中声明一块新的中断处理栈, 并将它加载到时钟中断的 IDT 中。在时间中断处调用进程切换函数, 再用 `as_handler` 宏重新定义中断处理函数, 它会生成一个函数, 先保存寄存器然后利用 `call` 指令调用原函数, 再恢复寄存器, 使用 `iretq` 指令返回。

3.2. 补全 `switch`、`save_current` 和 `switch_next` 函数的实现

```

pub fn switch(context: &mut ProcessContext) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        // FIXME: switch to the next process
        get_process_manager().save_current(context);

        let current = get_process_manager().current();
        if current.read().status() == ProgramStatus::Ready{
            get_process_manager().push_ready(current.pid());
        }
        get_process_manager().switch_next(context);
    });
}

pub fn save_current(&self, context: &ProcessContext) {
    // FIXME: update current process's tick count
    let cur = self.current();
    // FIXME: save current process's context
    cur.write().tick();
    cur.write().save(context);
    cur.write().pause();
}

pub fn switch_next(&self, context: &mut ProcessContext) -> ProcessId {
    // 1. 从就绪队列获取下一个进程
    let next_pid = loop {
        let pid = self.ready_queue.lock().pop_front();

        if let Some(pid) = pid {
            let proc = self.get_proc(&pid);

            // 2. 检查进程是否存在且就绪
            if let Some(proc) = proc {
                if proc.read().is_ready() {
                    break pid;
                }
            }
            // 如果进程不就绪，继续循环获取下一个
        } else {
            // 就绪队列空，返回当前进程 PID（无切换）
            return processor::get_pid();
        }
    };
};

```

4. 进程信息的获取

4.1. 环境变量

补全 env 函数:

```
pub fn env(key: &str) -> Option<String> {
    x86_64::instructions::interrupts::without_interrupts(|| {
        let current = get_process_manager().current();
        let inner = current.read();

        // Rust 自动插入 Deref 调用:
        // inner.env(key)
        // ↓ 自动解引用 RwLockReadGuard → &ProcessInner
        // (*inner).env(key)
        // ↓ ProcessInner 的 Deref 实现 → &ProcessData
        // ProcessData::env(&**inner, key)

        inner.env(key)
    })
}
```

4.2. 进程返回值

补全 wait 函数:

```
pub fn new_stack_test_thread() {
    let pid = spawn_kernel_thread(
        func::stack_test,
        alloc::string::String::from("stack"),
        None,
    );

    // wait for progress exit
    wait(pid);
} // 创建一个栈进程就要立马停下来等进程退出才结束创建过程

fn wait(pid: ProcessId) {
    loop {
        // FIXME: try to get the status of the process
        if let Some(proc) = get_process_manager().get_proc(&pid)
        { // HINT: it's better to use the exit code
            let EXIT_CODE = proc.read().exit_code();
            if EXIT_CODE.is_some() {
                break; // exit_code 不等于 none 就说明该进程已经退出了
            } else {
                x86_64::instructions::hlt();
            }
        } else {
            break;
        }
    }
}
```

5. 内核线程的创建

//在 `src/proc/mod.rs` 中定义的 `spawn_kernel_thread` 函数。它关闭中断，之后将函数转化为地址以使其能够赋值给 `rip` 寄存器，之后将进程的信息传递给 `ProcessManager`，使其创建所需进程。

```
pub fn spawn_kernel_thread(entry: fn() -> !, name: String, data:
Option<ProcessData>) -> ProcessId {
    x86_64::instructions::interrupts::without_interrupts(|| {
        let entry = VirtAddr::new(entry as usize as u64);
        get_process_manager().spawn_kernel_thread(entry, name, data)
    })
}

//src/proc/process.rs 中，根据内存布局预设和当前进程的 PID，为其分配初始栈空间。
pub fn spawn_kernel_thread(
    &self,
    entry: VirtAddr,
    name: String,
    proc_data: Option<ProcessData>,
) -> ProcessId {
    let kproc = self.get_proc(&KERNEL_PID).unwrap();
    let page_table = kproc.read().clone_page_table();
    let proc_vm = Some(ProcessVm::new(page_table));
    let proc = Process::new(name, Some(Arc::downgrade(&kproc)), proc_vm,
proc_data);

    // alloc stack for the new process base on pid
    let stack_top = proc.alloc_init_stack();
    let pid = proc.pid();
    // FIXME: set the stack frame
    proc.write().init_stack_frame(entry, stack_top);
    // FIXME: add to process map
    self.add_proc(pid, proc);
    // FIXME: push to ready queue
    self.push_ready(pid);
    // FIXME: return new process pid
    pid
}
```

6. 缺页异常的处理

在本实验设计中，实现为栈空间自动扩容来作为缺页异常的处理。

完善缺页异常的相关处理函数：

- 1.在 `ProcessManager` 中，检查缺页异常是否包含越权访问或其他非预期的错误码。
- 2.如果缺页异常是由于非预期异常导致的，或者缺页异常的地址不在当前进程的栈空间中，直接返回 `false`。

3.如果缺页异常的地址在当前进程的栈空间中，把缺页异常的处理委托给当前的进程。

- 需要为 `ProcessInner` 和 `ProcessVm` 添加用于分配新的栈、更新进程存储信息的函数。

4.在进程的缺页异常处理函数中：

分配新的页面、更新页表、更新进程数据中的栈信息。

`processmanager` 中有关页错误的处理函数：

```
pub fn handle_page_fault(&self, addr: VirtAddr, err_code: PageFaultErrorCode) ->
bool {
    // FIXME: handle page fault
    // 1. 检查保留位违规 - 硬件错误或严重问题
    if err_code.contains(PageFaultErrorCode::MALFORMED_TABLE) {
        return false;
    }
    // 2. 检查地址是否为空指针或接近空指针
    if addr.as_u64() < 0x1000 {
        return false;
    }
    // 3. 检查地址是否在有效的用户空间范围内
    if !is_canonical(addr.as_u64() as usize) {
        return false;
    }

    // 7. 检查是否在保护违规情况下访问内核空间
    let user_mode = err_code.contains(PageFaultErrorCode::USER_MODE);
    let protection =
err_code.contains(PageFaultErrorCode::PROTECTION_VIOLATION);

    // 用户态尝试访问内核空间
    if user_mode && addr.as_u64() >= 0xffff_8000_0000_0000 && protection {
        return false; // 非法访问内核空间 - 非预期
    }

    let current = self.current();
    let proc = current.read();

    drop(proc);
    if current.write().handle_page_fault(addr) {
        return true; // 成功处理 - 预期异常（如栈增长）
    }

    false
}
```

Page Fault 文件有关页处理的核心逻辑：

```

pub struct Stack {
    range: PageRange<Size4KiB>,
    usage: u64,
}

impl Stack {
    pub fn handle_page_fault(
        &mut self,
        addr: VirtAddr,
        mapper: MapperRef,
        alloc: FrameAllocatorRef,
    ) -> bool {
        if !self.is_on_stack(addr) {
            return false;
        }
        if let Err(m) = self.grow_stack(addr, mapper, alloc) {
            error!("Grow stack failed: {:?}", m);
            return false;
        }
        true
    }

    fn is_on_stack(&self, addr: VirtAddr) -> bool {
        let addr = addr.as_u64();
        let cur_stack_bot = self.range.start.start_address().as_u64();
        trace!("Current stack bot: {:#x}", cur_stack_bot);
        trace!("Address to access: {:#x}", addr);
        // Is it within the STACK_MAX_SIZE capacity?
        let max_stack_top = (cur_stack_bot & STACK_START_MASK) + STACK_MAX_SIZE;
        addr >= (cur_stack_bot & STACK_START_MASK) && addr < max_stack_top
    }

    fn grow_stack(
        &mut self,
        addr: VirtAddr,
        mapper: MapperRef,
        alloc: FrameAllocatorRef,
    ) -> Result<(), MapToError<Size4KiB>> {
        debug_assert!(self.is_on_stack(addr), "Address is not on stack.");
        // FIXME: grow stack for page fault
        let fault_page = Page::containing_address(addr);
        let current_bot = self.range.start;
        if fault_page >= current_bot {
            return Ok(());
        }
        let new_pages_count = current_bot - fault_page;
        let new_usage = self.usage + new_pages_count;
        if new_usage > STACK_MAX_PAGES {
            error!("Stack overflow: requested {} pages, max is {}",
                new_usage, STACK_MAX_PAGES);
            return Err(MapToError::FrameAllocationFailed);
        }
    }
}

```

```

        let flags = PageTableFlags::PRESENT | PageTableFlags::WRITABLE |
PageTableFlags::USER_ACCESSIBLE;

        for page in Page::range(fault_page, current_bot){
            let frame =
alloc.allocate_frame().ok_or(MapToError::FrameAllocationFailed)?;

            unsafe {
                mapper.map_to(page, frame, flags, alloc)?.flush();
            }
        }

        self.range = Page::range(fault_page, self.range.end);
        self.usage = new_usage;

        Ok(())
    }

    pub fn memory_usage(&self) -> u64 {
        self.usage * crate::memory::PAGE_SIZE
    }
}

```

processVM 中有关页错误处理的函数:

```

pub fn handle_page_fault(&mut self, addr: VirtAddr) -> bool {
    let mapper = &mut self.page_table.mapper();
    let alloc = &mut *get_frame_alloc_for_sure();

    self.stack.handle_page_fault(addr, mapper, alloc)
}

```

7. 进程的退出

实现进程的退出要完成如下几件事:

- 在进程退出时, 将进程的状态设置为 **Dead**, 并删除进程运行时需要的部分数据, 如 **ProcessData**。
- 确保进程不会被再次调度, 这可以在切换时添加检查、防止进程再次进入就绪队列来实现。
- 存储进程的返回值, 以便其他进程可以利用它来查询进程的退出状态。

8. 进程调度内核运行成果展示

以下是本次实验的终端输出结果:

```
[ INFO] [crates/kernel/src/utlis/logger.rs:11] Logger Initialized.
```

```

[ INFO] [crates/kernel/src/memory/address.rs:9] Physical Offset :
0xffff800000000000
[ INFO] [crates/kernel/src/memory/gdt.rs:34] Privilege Stack :
0xffffffff00000213c8-0xffffffff00000223c8
[ INFO] [crates/kernel/src/memory/gdt.rs:49] Privilege Stack :
0xffffffff00000223c8-0xffffffff00000233c8
[ INFO] [crates/kernel/src/memory/gdt.rs:61] Privilege Stack :
0xffffffff00000233c8-0xffffffff00000243c8
[ INFO] [crates/kernel/src/memory/gdt.rs:73] Privilege Stack :
0xffffffff00000243c8-0xffffffff00000253c8
[ INFO] [crates/kernel/src/memory/gdt.rs:85] Privilege Stack :
0xffffffff00000253c8-0xffffffff00000263c8
[ INFO] [crates/kernel/src/memory/gdt.rs:148] Kernel IST Size : 12.000 KiB
[ INFO] [crates/kernel/src/memory/gdt.rs:150] GDT Initialized.
[DEBUG] [crates/kernel/src/memory/allocator.rs:26] Kernel Heap :
0xffffffff00000263c8-0xffffffff00020263c8
[ INFO] [crates/kernel/src/memory/allocator.rs:33] Kernel Heap Size : 32.000 MiB
[ INFO] [crates/kernel/src/memory/allocator.rs:35] Kernel Heap Initialized.
[TRACE] [crates/kernel/src/interrupt/apic/ioapic.rs:74] Enable IOApic: IRQ=4,
CPU=0
[TRACE] [crates/kernel/src/interrupt/apic/ioapic.rs:74] Enable IOApic: IRQ=0,
CPU=0
[ INFO] [crates/kernel/src/interrupt/mod.rs:46] Interrupts Initialized.
[ INFO] [crates/kernel/src/memory/mod.rs:26] Physical Memory : 12.093 GiB
[ INFO] [crates/kernel/src/memory/mod.rs:29] Free Usable Memory : 10.906 MiB
[ INFO] [crates/kernel/src/memory/mod.rs:38] Frame Allocator initialized.
[TRACE] [crates/kernel/src/proc/mod.rs:36] Init kernel vm: ProcessVm {
    stack: Stack {
        top: 0xffffffff01ffffff00,
        bot: 0xffffffff01ffe00000,
    },
    memory_usage: "2 MiB",
    page_table: PageTable {
        addr: PhysFrame[4KiB](0x5801000),
        flags: Cr3Flags(
            0x0,
        ),
    },
}
[TRACE] [crates/kernel/src/proc/process.rs:72] New process kernel#1 created.

```

```

[TRACE] [crates/kernel/src/proc/manager.rs:43] Init Process {
  pid: 1,
  name: "kernel",
  parent: None,
  status: Running,
  ticks_passed: 0,
  children: Map {
    iter: Iter(
      [],
    ),
  },
  status: Running,
  context: StackFrame {
    stack_top: VirtAddr(
      0x2000,
    ),
    cpu_flags: RFlags(
      0x0,
    ),
    instruction_pointer: VirtAddr(
      0x1000,
    ),
    regs: Registers
    r15: 0x0000000000000000, r14: 0x0000000000000000, r13:
0x0000000000000000,
    r12: 0x0000000000000000, r11: 0x0000000000000000, r10:
0x0000000000000000,
    r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi:
0x0000000000000000,
    rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx:
0x0000000000000000,
    rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp:
0x0000000000000000,
  },
  vm: Some(
    ProcessVm {
      stack: Stack {
        top: 0xffffffff01ffff00,
        bot: 0xffffffff01ffe00000,
      },
      memory_usage: "2 MiB",
      page_table: PageTable {
        addr: PhysFrame[4KiB](0x5801000),
        flags: Cr3Flags(
          0x0,
        ),
      },
    ),
  ),
}

```



```

[ INFO] [crates/kernel/src/proc/mod.rs:64] Process Manager Initialized.
[ INFO] [crates/kernel/src/lib.rs:53] Interrupts Enabled.
[ INFO] [crates/kernel/src/lib.rs:55] YYOS initialized.
[ INFO] [crates/kernel/src/main.rs:15] 开始创建进程:
[TRACE] [crates/kernel/src/proc/process.rs:72] New process #0_test#2 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB]
(0x3ffdffffff000), end: Page[4KiB] (0x3ffe00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame:
InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0xffffffff000000c0a0,
    ),
    code_segment: SegmentSelector {
        index: 1,
        rpl: Ring0,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ffdffffff8,
    ),
    stack_segment: SegmentSelector {
        index: 2,
        rpl: Ring0,
    },
}

[TRACE] [crates/kernel/src/proc/process.rs:72] New process #1_test#3 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB]
(0x3ffcffffff000), end: Page[4KiB] (0x3ffd00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame:
InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0xffffffff000000c0a0,
    ),
    code_segment: SegmentSelector {
        index: 1,
        rpl: Ring0,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ffcffffff8,
    ),
    stack_segment: SegmentSelector {
        index: 2,
        rpl: Ring0,
    },
}

```

```

[TRACE] [crates/kernel/src/proc/process.rs:72] New process #2_test#4 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB]
(0x3ffbffff000), end: Page[4KiB] (0x3ffc00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame:
InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0xffffffff000000c0a0,
    ),
    code_segment: SegmentSelector {
        index: 1,
        rpl: Ring0,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ffbffffffffff8,
    ),
    stack_segment: SegmentSelector {
        index: 2,
        rpl: Ring0,
    },
}

[TRACE] [crates/kernel/src/proc/process.rs:72] New process #3_test#5 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB]
(0x3ffaafffff000), end: Page[4KiB] (0x3ffb00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame:
InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0xffffffff000000c0a0,
    ),
    code_segment: SegmentSelector {
        index: 1,
        rpl: Ring0,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ffaaffffffffffff8,
    ),
    stack_segment: SegmentSelector {
        index: 2,
        rpl: Ring0,
    },
}

```

```

[TRACE] [crates/kernel/src/proc/process.rs:72] New process #4_test#6 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB]
(0x3ff9ffff000), end: Page[4KiB] (0x3ffa00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame:
InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0xffffffff000000c0a0,
    ),
    code_segment: SegmentSelector {
        index: 1,
        rpl: Ring0,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ff9ffffffffff8,
    ),
    stack_segment: SegmentSelector {
        index: 2,
        rpl: Ring0,
    },
}
[ INFO] [crates/kernel/src/main.rs:20] Created 5 test threads, scheduler is
running...
[>] ps
  PID | PPID | Process Name | Ticks | Status
#  2 | #  1 | #0_test      |    661 | Ready
#  1 | #  0 | kernel       |    666 | Running
#  4 | #  1 | #2_test      |    659 | Ready
#  5 | #  1 | #3_test      |    658 | Ready
#  3 | #  1 | #1_test      |    660 | Ready
#  6 | #  1 | #4_test      |    657 | Ready
Heap   : 7.281 KiB used / 31.993 MiB free / 32.000 MiB total
Queue  : [2, 3, 4, 5, 6]
CPUs   : [0: 1]
[>] test
[TRACE] [crates/kernel/src/proc/process.rs:72] New process #5_test#7 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB]
(0x3ff8ffff000), end: Page[4KiB] (0x3ff900000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame:
InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0xffffffff000000c0a0,
    ),
    code_segment: SegmentSelector {
        index: 1,
        rpl: Ring0,
    },
}

```

```

    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ff8ffffffff8,
    ),
    stack_segment: SegmentSelector {
        index: 2,
        rpl: Ring0,
    },
}

[>] ps
  PID | PPID | Process Name | Ticks | Status
#  2 | #   1 | #0_test      | 1353 | Ready
#  1 | #   0 | kernel       | 1358 | Running
#  4 | #   1 | #2_test      | 1351 | Ready
#  5 | #   1 | #3_test      | 1350 | Ready
#  3 | #   1 | #1_test      | 1352 | Ready
#  6 | #   1 | #4_test      | 1349 | Ready
#  7 | #   1 | #5_test      | 287  | Ready
Heap   : 7.977 KiB used / 31.992 MiB free / 32.000 MiB total
Queue  : [2, 3, 4, 5, 6, 7]
CPUs   : [0: 1]

[>] stack
[TRACE] [crates/kernel/src/proc/process.rs:72] New process stack#8 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB]
(0x3ff7ffffff000), end: Page[4KiB](0x3ff800000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame:
InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0xffffffff000000c090,
    ),
    code_segment: SegmentSelector {
        index: 1,
        rpl: Ring0,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ff7ffffff8,
    ),
    stack_segment: SegmentSelector {
        index: 2,
        rpl: Ring0,
    },
}

```

```

[TRACE] [crates/kernel/src/proc/vm/stack.rs:104] Current stack bot:
0x3ff7ffff000
[TRACE] [crates/kernel/src/proc/vm/stack.rs:105] Address to access:
0x3ff7ffff7f88
Huge stack testing...
0x000 == 0x000
0x100 == 0x100
0x200 == 0x200
0x300 == 0x300
0x400 == 0x400
0x500 == 0x500
0x600 == 0x600
0x700 == 0x700
0x800 == 0x800
0x900 == 0x900
0xa00 == 0xa00
0xb00 == 0xb00
0xc00 == 0xc00
0xd00 == 0xd00
0xe00 == 0xe00
0xf00 == 0xf00
[TRACE] [crates/kernel/src/proc/manager.rs:202] Kill Process {
  pid: 8,
  name: "stack",
  parent: Some(
    1,
  ),
  status: Running,
  ticks_passed: 3,
  children: Map {
    iter: Iter(
      [],
    ),
  },
  status: Running,
  context: StackFrame {
    stack_top: VirtAddr(
      0x3ff7ffff7f68,
    ),
    cpu_flags: RFlags(
      IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG | 0x2,
    ),
    instruction_pointer: VirtAddr(
      0xffffffff0000005c46,
    ),
  },
}

```

```

regs: Registers
r15: 0xffffffff0000020030, r14: 0xffffffff0000001d13, r13:
0xffffffff0000018af0,
r12: 0xffffffff0000018af0, r11: 0x0000000000000000, r10:
0x0000000000000002,
r9 : 0x000000000000000d, r8 : 0x000000000000003fd, rdi:
0x000000000000003f8,
rsi: 0xffffffff0000001d29, rdx: 0x000000000000003f8, rcx:
0xffffffff0000001d29,
rbx: 0x00003ff7ffffff90, rax: 0x0000000000000000, rbp:
0x00003ff7ffff7f80,
},
vm: Some(
  ProcessVm {
    stack: Stack {
      top: 0x3ff800000000,
      bot: 0x3ff7ffff7000,
    },
    memory_usage: "36 KiB",
    page_table: PageTable {
      addr: PhysFrame[4KiB](0x1f000),
      flags: Cr3Flags(
        0x0,
      ),
    },
  },
),
}
[DEBUG] [crates/kernel/src/proc/process.rs:84] Killing process stack#8 with ret
code: 0
[TRACE] [crates/kernel/src/proc/process.rs:189] Process stack killed with exit
code 0
[>] ps
  PID | PPID | Process Name | Ticks | Status
#  5 | #  1 | #3_test      | 1930 | Ready
#  1 | #  0 | kernel       | 1938 | Running
#  2 | #  1 | #0_test      | 1933 | Ready
#  4 | #  1 | #2_test      | 1931 | Ready
#  6 | #  1 | #4_test      | 1929 | Ready
#  3 | #  1 | #1_test      | 1932 | Ready
#  7 | #  1 | #5_test      | 867  | Ready
Heap   : 8.438 KiB used / 31.992 MiB free / 32.000 MiB total
Queue  : [2, 3, 4, 5, 6, 7]
CPUs   : [0: 1]
[>] test
[TRACE] [crates/kernel/src/proc/process.rs:72] New process #6_test#9 created.
[TRACE] [crates/elf/src/lib.rs:51] Page Range: PageRange { start: Page[4KiB]
(0x3ff6fffff000), end: Page[4KiB](0x3ff700000000) }(1)

```

```
[TRACE] [crates/kernel/src/proc/context.rs:55] Init stack frame:
InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0xffffffff000000c0a0,
    ),
    code_segment: SegmentSelector {
        index: 1,
        rpl: Ring0,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ff6ffffffff8,
    ),
    stack_segment: SegmentSelector {
        index: 2,
        rpl: Ring0,
    },
}
[>] ps
  PID | PPID | Process Name | Ticks | Status
#  5 | #  1 | #3_test      |    2664 | Ready
#  1 | #  0 | kernel       |    2672 | Running
#  2 | #  1 | #0_test      |    2667 | Ready
#  9 | #  1 | #6_test      |     396 | Ready
#  4 | #  1 | #2_test      |    2665 | Ready
#  6 | #  1 | #4_test      |    2663 | Ready
#  3 | #  1 | #1_test      |    2666 | Ready
#  7 | #  1 | #5_test      |    1601 | Ready
Heap   : 9.500 KiB used / 31.991 MiB free / 32.000 MiB total
Queue  : [2, 3, 4, 5, 6, 7, 9]
CPUs   : [0: 1]
[>] QEMU: Terminated
```

9. 实验结果分析

本次实验成功展示了 *YatSenOS* (YYOS) 内核的初始化及多任务管理能力。以下是对实验输出结果的详细描述：

1. 内核环境初始化：系统成功进入内核入口，完成了串口 (Serial)、日志系统 (Logger) 及全局描述符表 (GDT) 的初始化。内核堆空间 (Kernel Heap) 设置为 32 MiB，物理内存识别为 12.093 GiB。中断控制器 (IOAPIC) 正确启用了 IRQ 0 (时钟) 与 IRQ 4 (串口) 中断。
2. 多任务创建与调度：
 - 内核成功创建了初始进程 kernel (PID 1)，并初始化其虚拟内存空间。

- 通过 Shell 指令多次触发测试线程创建，系统成功孵化了从 `#0_test` 到 `#6_test` 的多个内核线程。
 - `ps` 命令显示所有线程状态正常，`Ticks` 计数持续增长，证明基于时钟中断的抢占式调度器运行稳定，就绪队列（Queue）管理正确。
3. 内存管理与大栈测试(**Huge Stack Test**): 在执行 `stack` 命令时, 系统创建了 `stack (PID 8)` 进程。实验记录显示:
- 按需分页: 当进程尝试访问 `0x3ff7ffff7f88` 等超出初始物理映射的地址时, 触发了页错误处理机制。
 - 栈动态增长: 系统成功捕获异常并分配物理页, 输出了从 `0x000` 到 `0xf00` 的连续内存访问记录, 验证了栈空间的自动扩展功能。
4. 进程回收机制: `stack` 进程在完成测试后, 通过内核函数正确触发了进程终止逻辑。日志显示 `Kill Process` 动作清理了该进程的上下文及虚拟内存资源, 返回退出码 `0`, PID 被系统回收。

结论: 实验输出结果符合预期。内核已具备稳定的物理/虚拟内存管理、可扩展的内核栈机制、以及基本的进程生命周期管理功能。

10. 思考题

10.1. 为什么在初始化进程管理器时需要将它置为正在运行的状态? 能否通过将它置为就绪状态并放入就绪队列来实现? 这样的实现可能会遇到什么问题?

进程管理器初始状态设计分析:

在初始化进程管理器时, 将第一个进程(如 `kernel` 进程)直接置为 **Running** 状态而非 **Ready** 状态, 是基于操作系统底层引导逻辑与硬件约束的必然选择。以下从三个维度进行深度分析:

10.1.1. 现状描述的客观性

内核在执行 `rust_main` 逻辑时, 物理 CPU 已经在运行当前的代码指令。此时创建 `kernel` 进程并将其设为 **Running**, 是对当前物理事实的软件同步:

- 若设为 **Ready**, 则意味着软件层面认为 CPU 处于空闲或尚未开始该任务, 这会导致逻辑上的自相矛盾。
- 只有置为 **Running**, 进程管理器才能正确地将当前的执行上下文(寄存器、栈)与 PID 1 绑定。

10.1.2. 避免“引导死锁”(Bootstrapping Deadlock)

若尝试将初始进程放入就绪队列实现, 会面临“先有鸡还是先有蛋”的问题:

- 中断依赖性: 调度通常依赖时钟中断触发。若当前没有进程处于 **Running** 状态, 当中断发生时, 硬件尝试压栈保存现场将失去合法的栈支撑, 导致系统触发 **Double Fault** 重启。

- 切换逻辑局限: 标准的上下文切换函数 `switch(from, to)` 要求必须有一个“当前正在运行”的来源 (`from`)。如果没有 `Running` 进程, 该函数将因找不到有效的旧栈指针而崩溃。

10.1.3. 可能遇到的风险与问题

如果强行通过“放入就绪队列再启动”的模式实现, 可能会遇到以下技术难题:

潜在问题	详细描述
上下文丢失	初始化代码中包含大量“仅限一次”的硬件配置, 若在配置中途被切换走, 可能导致硬件状态不一致。
栈空间非法	引导阶段使用的是临时栈, 若不立即将其合法化为运行进程, 该段空间可能被后续分配逻辑污染。
空队列恐慌	若就绪队列因某种异常未及时更新, 调度器可能从空队列中抓取任务, 导致内核进入不可恢复的 Panic 状态。

结论: 将初始进程直接标记为 `Running`, 是为了建立一个合法的执行基准, 使得后续的“中断 -> 调度 -> 切换”链条能够平滑启动, 是保证内核从静态引导转向动态多任务运行的关键。

10.2. 在 `src/proc/process.rs` 中, 有两次实现 `Deref` 和一次实现 `DerefMut` 的代码, 它们分别是为了什么? 使用这种方式提供了什么便利?

在 `[ process.rs ](yyos/crates/kernel/src/proc/process.rs)` 中, 存在以下 `Deref/DerefMut` 实现:
第一次 `Deref` (`Process` $\rightarrow$ `RwLock`):

```
impl Deref for Process {
    type Target = RwLock<ProcessInner>;
    fn deref(&self) -> &Self::Target { &self.inner }
}
```

允许直接调用 `process.read()` / `process.write()` 而非 `process.inner.read()`。

第二次 `Deref` (`ProcessInner` $\rightarrow$ `ProcessData`):

```
impl Deref for ProcessInner {
    type Target = ProcessData;
    fn deref(&self) -> &Self::Target {
        self.proc_data.as_ref().expect(...)
    }
}
```

允许直接访问 `ProcessData` 的方法, 如 `inner.env(key)`。

`DerefMut` (`ProcessInner` $\rightarrow$ `ProcessData`):

```
impl DerefMut for ProcessInner {
    fn deref_mut(&mut self) -> &mut Self::Target {
        self.proc_data.as_mut().expect(...)
    }
}
```

允许可变访问 `ProcessData`，如 `inner.set_env(key, val)`。

这种链式 `Deref` 设计实现了「智能指针」语义，在保持封装性的同时简化了多层嵌套结构的访问语法，体现了 Rust 中「组合优于继承」的典型实践。

10.3. 中断的处理过程默认是不切换栈的，即在中断发生前的栈上继续处理中断过程，为什么在处理缺页异常和时钟中断时需要切换栈？如果不为它们切换栈会分别带来哪些问题？请假设具体的场景、或通过实际尝试进行回答。

缺页异常与时钟中断中的栈切换必要性分析：

在 [x86_64] 架构下，中断发生时默认不切换栈（除非特权级发生改变）。但在处理缺页异常（Page Fault）和时钟中断（Timer Interrupt）时，内核通常会显式配置 [IST (Interrupt Stack Table)] 或手动切换栈。其必要性分析如下：

10.3.1. 处理缺页异常（Page Fault）时的栈切换

原因：防止内核栈溢出导致的双重异常（Double Fault）

缺页异常可能在任何地方发生，包括内核自身正在进行深度函数调用或在栈上分配大型局部变量时。

- 假设场景：内核函数正在执行一个深递归操作，此时栈指针 [RSP] 距离页边界仅剩 [16 字节]。此时函数尝试压入一个新变量，触发了缺页异常。
- 不切换栈的问题：若不切换栈，CPU 会尝试在当前已经触底的 [RSP] 下方压入 [Interrupt Stack Frame] (约 40 字节)。由于此时栈已经没有空间且触发异常的正是“栈写入”，CPU 会在处理“缺页异常”的过程中再次触发“缺页异常”，最终导致 [Double Fault]，硬件直接重启，内核无法打印任何调试信息。
- 解决方案：通过 [IST] 为缺页异常分配专门的、干净的栈空间，确保即便内核栈已经爆满，异常处理程序依然能安全运行并尝试修复映射或优雅地 Panic。

10.3.2. 处理时钟中断（Timer Interrupt）时的栈切换

原因：支持多任务并发与内核抢占（Preemption）

时钟中断是任务调度的引擎。处理它时切换栈不是因为硬件限制，而是为了软件架构的解耦。

- 假设场景：进程 A 正在内核态运行，时钟中断发生。此时调度器决定切换到进程 B 运行。

- 不切换栈的问题：若复用当前栈，时钟中断的处理逻辑、寄存器现场以及后续调度器的局部变量都会压在进程 A 的内核栈上。
 1. 逻辑耦合：进程 A 的栈将不得不保留“调度器”的状态，导致栈空间碎片化。
 2. 嵌套风险：如果在处理时钟中断（还在 A 的栈上）时又发生了一个高优先级中断，栈的深度将变得不可控。
 3. 隔离性差：若所有 CPU 的时钟处理都混杂在当前进程栈中，内核将难以实现统一的中断处理模型。
- 实际尝试观察：在 [YYOS] 实验中，如果不为中断配置独立的 [Privilege Stack] 或 [IST]，在频繁输入指令（Shell）同时触发时钟中断时，你会观察到内核经常报出 [Stack Overflow] 或者上下文恢复后寄存器数值错乱，这正是因为不同任务的执行流在同一个物理栈上发生了“踩踏”。

10.3.3. 总结对比

中断类型	不切换栈的后果	切换栈带来的收益
缺页异常	导致 [Double Fault], 系统直接死锁/重启。	保证异常处理逻辑始终有可用栈空间。
时钟中断	任务状态污染，增加栈溢出风险，难以实现内核抢占。	实现中断上下文与进程上下文的解耦，支持平滑调度。

结论：栈切换是内核“防御性编程”的一种体现。对于异常，它是为了生存（避免硬件崩溃）；对于时钟中断，它是为了秩序（支持复杂的任务切换）。